

محمدرضا شریفی

۴۰۱۱۰۱۹۳۲

فاز ۲

طراحی سیستم های میکروپروسسوری

پردازنده ی لپتاپ intel core i7 نسل ۱۱ میباشد و دارای ۴ core میباشد.

$$\text{size of } L_1 \text{ cache} = \frac{320}{4} = 80Kb$$

$$\text{size of } L_2 \text{ cache} = \frac{5}{4} = 1.25 Mb$$

$$\text{size of } L_3 \text{ cache} = 12 Mb$$

زمان های محاسبه شده یا میانگین آزمایش ها هستند و یا بیشترین تکرار را میان آزمایش ها داشتند.

۱-۲

ابتدا در این قسمت با کد زیر ماتریکس را ترانواده میکنیم:

```
// Transpose matrix B into b_t (reading sequentially using pointer)
register double *p_b = b;
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++, p_b++) {
        b_t[j * n + i] = *p_b;
    }
}
```

سپس ضرب را به شکل زیر پیاده سازی میکنیم:

```
// Multiply using Phase 1 optimizations (pointers & registers)
register double sum;
register double *pa, *pb_t;
for (i = 0; i < n; i++, a += n) {
    for (j = 0; j < n; j++, c++) {
        sum = 0.0;
        pb_t = b_t + (j * n); // Point to start of row j in b_t
        for (k = 0, pa = a; k < n; k++, pa++, pb_t++) {
            sum += (*pa) * (*pb_t); // Pure pointer traversal
        }
        *c = sum;
    }
}
_aligned_free(b_t);
```

n	matrix_mult_ptr_reg	matrix_mult_transpose	Gain
512	0.25 s	0.12 s	2.08
1024	2.30s	1s	2.30
2048	44.2s	8.75s	5
4096	537s	69s	7.78

روند کلی به این شکل است که به ازای **n** های بالاتر گین بیشتری میگیریم زیرا که تعداد محاسبات بیشتر شده و نیاز به دسترسی به داده ها نیز افزایش یافته است بنابراین بیشتر نیاز است به سراغ کش برویم و ترانواده بهتر از کش استفاده میکند.

N=1024:

```
Do you want to run matrix_mult_ptr_reg (y/n)? y
Execution time of matrix_mult_ptr_reg = 2.26 s

Do you want to run matrix_mult_ptr_no_reg (y/n)? n

Do you want to run matrix_mult_transpose (y/n)? y
Execution time of matrix_mult_transpose = 1.00 s
```

N=2048:

```
Do you want to run matrix_mult_ptr_reg (y/n)? y
Execution time of matrix_mult_ptr_reg = 44.48 s

Do you want to run matrix_mult_ptr_no_reg (y/n)? n

Do you want to run matrix_mult_transpose (y/n)? y
Execution time of matrix_mult_transpose = 8.93 s
```

N=4096:

```
Do you want to run matrix_mult_ptr_reg (y/n)? y
Execution time of matrix_mult_ptr_reg = 537.25 s

Do you want to run matrix_mult_ptr_no_reg (y/n)? n

Do you want to run matrix_mult_transpose (y/n)? y
Execution time of matrix_mult_transpose = 69.11 s
```

همانند شکل سوال کد زیر را با کمک لوپ های تودرتو پیاده سازی میکنیم.

```
void matrix_mult_block(int n, int block_size, double* a, double* b, double* c) {
    register int i, j, k, i_inner, j_inner, k_inner;
    register double a_val;
    register double *pc, *pb;
    // Initialize the result matrix C to 0 using pointers
    register double *pc_init = c;
    for (i = 0; i < n * n; i++, pc_init++) {
        *pc_init = 0.0;
    }
    // Outer loops iterating over blocks
    for (i = 0; i < n; i += block_size) {
        for (k = 0; k < n; k += block_size) {
            for (j = 0; j < n; j += block_size) {
                // Cache loop limits into registers
                register int i_limit = MIN(i + block_size, n);
                register int k_limit = MIN(k + block_size, n);
                register int j_limit = MIN(j + block_size, n);
                for (i_inner = i; i_inner < i_limit; i_inner++) {
                    for (k_inner = k; k_inner < k_limit; k_inner++) {
                        a_val = a[i_inner * n + k_inner];
                        // Set up pointers for the innermost loop
                        pc = c + (i_inner * n + j);
                        pb = b + (k_inner * n + j);

                        // Innermost loop: pure pointer increment
                        for (j_inner = j; j_inner < j_limit; j_inner++, pc++, pb++) {
                            *pc += a_val * (*pb);
                        }
                    }
                }
            }
        }
    }
}
```

n	Block size	matrix_mult_block	matrix_mult_ptr_reg	Gain
512	8	0.10	0.25	2.5
512	16	0.09		2.77
512	32	0.09		2.77
512	64	0.08		3.125
512	128	0.08		3.125
1024	8	0.80	2.30	2.875
1024	16	0.72		3.19
1024	32	0.77		2.98
1024	64	0.63		3.65
1024	128	0.56		4.1
1024	256	0.57		4.03
1024	512	0.56		4.10
2048	8	6.52	44.2	6.77
2048	16	6.14		7.19
2048	32	6.3		7.01
2048	64	5.26		8.40
2048	128	4.84		9.13
2048	256	4.50		9.82
4096	8	82	537	6.54
4096	16	68.5		7.83
4096	32	70		7.67
4096	64	57.7		9.3
4096	128	51		10.52
4096	256	44.1		12.17
4096	512	46		11.67

توجیه نتایج بر اساس ساختار حافظه cache :

رفتار مشاهده شده در زمان‌های اجرا برای اندازه‌های مختلف بلاک کاملاً با مشخصات سخت‌افزاری پردازنده تطابق دارد و به شرح زیر توجیه می‌شود:

۱. چرا بلوک‌های بسیار کوچک (۸ و ۱۶) با وجود سرعت بالای کش L1، کندتر هستند؟

بار اضافی حلقه‌ها (Loop Overhead): در الگوریتم بلوکی، تعداد حلقه‌های تودرتو (از ۳ به ۶ حلقه) افزایش می‌یابد. در بلوک‌های بسیار کوچک مانند ۸*۸، پردازنده زمان بسیار زیادی را صرف مدیریت پرش‌ها و کنترل حلقه‌های بیرونی می‌کند که این زمان هدررفته، مزیت سرعت کش L1 را خنثی می‌سازد.

عدم استفاده کامل از ظرفیت L1: یک بلوک ۸*۸ از نوع double تنها ۵۱۲ بایت حجم دارد. قرار دادن این حجم ناچیز در یک کش ۸۰ کیلوبایتی به معنای هدر دادن بخش عمده‌ای از ظرفیت این حافظه فوق‌سریع است. هدف از بلوک‌بندی، پر کردن حداکثری ظرفیت کش بدون سرریز شدن آن است.

۲. تعامل بهینه اندازه بلوک‌ها با سطوح مختلف Cache:

کش L1 (ظرفیت ۸۰ kb): برای یک بلوک ۶۴*۶۴ (با داده‌های ۸ بایتی double)، هر بلوک حدود ۳۲.۷ کیلوبایت فضا نیاز دارد. از آنجا که در ضرب ماتریس به ۳ بلوک در لحظه نیاز داریم، مجموعاً به حدود ۹۸ کیلو بایت فضا نیاز است که تطابق بسیار خوبی با ظرفیت کش L1 دارد و دقیقاً از همین نقطه بهبود عملکرد چشمگیر می‌شود.

کش L2 (ظرفیت ۱.۲۵Mb): بلوک‌های ۱۲۸*۱۲۸ (در مجموع ۳۹۳ کیلوبایت برای ۳ بلوک) و ۲۵۶*۲۵۶ (در مجموع حدود ۱.۵ مگابایت برای ۳ بلوک) به خوبی در سطح L2 و بخشی از L3 جای می‌گیرند و بهترین تعادل را ایجاد می‌کنند.

افت عملکرد در بلوک‌های بیش از حد بزرگ: برای بلوک سایز ۵۱۲، حجم هر بلوک به حدود ۲ مگابایت می‌رسد که از ظرفیت کش L2 بیشتر است. این سرریز باعث وابستگی اجباری به کش کندتر L3 شده و در نتیجه زمان اجرا افزایش می‌یابد.

۳. چرا در ماتریس‌های بزرگتر ($n = 4096$) بلوک‌های ۱۲۸ یا ۲۵۶ را عملکرد بهتر می‌باشد، اما در ماتریس‌های کوچکتر بلوک ۱۲۸ عملکرد بهتری دارد؟

این مسئله به مفهوم Arithmetic Intensity و گلوگاه حافظه اصلی یا همان رم برمی‌گردد:

ماتریس‌های بزرگ ($n = 4096$): حجم این ماتریس‌ها بسیار بزرگ است و انتقال داده از رم به پردازنده بسیار کند انجام می‌شود. در این حالت، وقتی یک بلوک وارد کش L2/L3 می‌شود، باید تا حد امکان بیشترین محاسبات روی آن انجام شود تا کندی رم جبران شود. بلوک‌های بزرگتر نظیر ۲۵۶ این امکان را فراهم می‌کنند که به ازای هر بار واکنشی داده از حافظه گُند، محاسبات بیشتری انجام گیرد.

ماتریس‌های کوچکتر ($n = 1024$): کل حجم ماتریس کوچکتر است و بخش زیادی از آن به راحتی در کش L3 جای می‌گیرد. در این حالت، تاخیر حافظه رم کمتر گلوگاه ایجاد می‌کند، بنابراین بلوک سایز ۱۲۸ بهینه‌ترین حالت است؛ زیرا هم داده‌ها را در کش‌های فوق‌سریع L1 و L2 نگه می‌دارد و هم بار اضافی حلقه‌ها را در سطح معقولی حفظ می‌کند.